TECHNISCHE HOCHSCHULE MITTELHESSEN | CAMPUS FRIEDBERG | MND Mathematik, Naturwissenschaften und Datenverarbeitung

Microservices

Dokumentation

SmartQueryBI

Mehrschichtige Caching-Architekturen in modernen BI-Systemen

Verfasser der Arbeit

Kamal Badawi - 5236028

Betreuer

Torsten Steinmüller

Wintersemester 2025/26

05. Dezember 2025

Inhaltsverzeichnis

1. Einleitung ... 4
1.1. Motivation ... 4
1.2. Idee ... 4
1.3. Zielsetzung ... 5
1.4. Relevanz ... 5
2. Datenmodell ... 5
2.1. Faktentabelle: sales_fact ... 6
2.2. Dimensionstabelle: product_dim ... 6
2.3. Dimensionstabelle: employee_dim ... 7
2.4. Dimensionstabelle: store_dim ... 7
2.5. Dimensionstabelle: customer_dim ... 8
2.6. Dimensionstabelle: date_dim ... 8
2.7. Zusammenfassung ER-Modell ... 9
3. LLM-Einsatz ... 9
3.1. Vorteile von Gemini 2.5 Pro ... 10
3.2. Einsatzszenarien ... 10
4. Caching-Strategien in SmartQueryBI ... 10
4.1. Client-Side Caching ... 10
4.1.1. Funktionsweise ... 10
4.1.2. Implementierung ... 11
4.1.3. Vorteile ... 11
4.1.4. Nachteile ... 11
4.1.5. Einsatzszenarien ... 11
4.2. Server-Side LRU Cache ... 11
4.2.1. Funktionsweise ... 11
4.2.2. Implementierung ... 11
4.2.3. Vorteile ... 12
4.2.4. Nachteile ... 12
4.2.5. Einsatzszenarien ... 12
4.3. Redis Distributed Cache ... 12
4.3.1. Funktionsweise ... 12
4.3.2. Implementierung ... 12
4.3.3. Vorteile ... 13

4.3.4. Nachteile 13
4.3.5. Einsatzszenarien 13
4.4. Vergleich der Caching-Ebenen 13
4.5. Best Practices 14
4.6. Architekturdiagramm (Cache-Hierarchie) 14
5. Fazit 15

1. Einleitung

1.1. Motivation

In modernen Unternehmen wachsen Datenmengen exponentiell. Business Intelligence (BI)-Systeme stehen vor der Herausforderung, große Datenbestände effizient zu verarbeiten und gleichzeitig interaktive Dashboards, Ad-hoc-Analysen und Echtzeit-Reports für Endnutzer bereitzustellen. Traditionelle Ansätze stoßen dabei an Grenzen: Häufige, wiederholte Abfragen an die Datenbank verursachen hohe Latenzen, steigende Kosten und eine zunehmende Serverbelastung.

Die Integration von Large Language Models (LLMs) wie Gemini 2.5 Pro eröffnet neue Möglichkeiten: Nutzer können natürliche Sprache verwenden, um komplexe Datenabfragen zu formulieren, während das System automatisch die optimale SQL-Query erstellt und die Ergebnisse in interaktive Visualisierungen transformiert.

Allerdings ist der direkte LLM-Einsatz ressourcenintensiv und teuer. Hier kommt intelligentes Caching ins Spiel: Durch temporäre Zwischenspeicherung von Ergebnissen lassen sich Antwortzeiten verkürzen, Kosten senken und Serverlast reduzieren.

1.2. Idee

SmartQueryBI ist eine AI-basierte BI-Plattform, die LLMs mit einem mehrschichtigen Caching-System kombiniert. Die Idee besteht darin, häufige und wiederkehrende Datenanfragen effizient zwischenzuspeichern, um sowohl Performance als auch Skalierbarkeit zu optimieren.

Die zentrale Idee umfasst:

1. Natürliche Sprachabfragen: Endnutzer beschreiben ihre Analysebedürfnisse in einfacher Sprache.
2. Automatische Query-Generierung: LLM übersetzt Text in SQL-Queries, die für das Star-Schema der Datenbank optimiert sind.
3. Interaktive Visualisierung: Ergebnisse werden automatisch in Nivo.js-kompatible Charts umgewandelt.
4. Mehrschichtiges Caching: Client-Side, Server-Side LRU und Redis Distributed Caching minimieren Latenz, Kosten und Last.

1.3. Zielsetzung

Ziel der vorliegenden Arbeit ist es, die Implementierung und den Nutzen eines mehrschichtigen Caching-Systems in einer modernen BI-Plattform detailliert zu dokumentieren und zu analysieren. Die Arbeit verfolgt dabei folgende Teilziele:

1. Darstellung des Datenmodells: Erläuterung des Sales Star Schemas als Grundlage für analytische Abfragen.
2. Analyse der Caching-Ebenen: Untersuchung von Client-, Server- und Redis-Caching hinsichtlich Performance, Skalierbarkeit, Persistenz und Einsatzszenarien.
3. Integration von LLMs: Beschreibung der Rolle von Gemini 2.5 Pro für Query-Generierung und Visualisierungsauswahl.
4. Best Practices und Empfehlungen: Aufzeigen von Strategien für TTL-Management, User-Isolation, Cache Invalidation und Monitoring.
5. Evaluation der Effizienz: Vergleich der Vorteile und Limitationen jeder Caching-Ebene in praxisnahen Szenarien.

1.4. Relevanz

Die vorliegende Arbeit zeigt, wie AI-basierte BI-Plattformen durch ein strategisch implementiertes Caching-System:

- Schnelle, interaktive Analysen ermöglichen
- Ressourcenverbrauch und Betriebskosten reduzieren
- Skalierbarkeit und Stabilität für große Nutzerzahlen gewährleisten

Damit leistet SmartQueryBI einen Beitrag zur Effizienzsteigerung in datengetriebenen Unternehmen und demonstriert die Kombination von LLM-Technologie mit intelligenter Cache-Architektur als praxisnahe Lösung für moderne BI-Anforderungen.

2. Datenmodell

Das BI-System basiert auf einem klassischen Star Schema, das Analysen und Aggregationen optimiert.

Zentrale Faktentabelle: sales_fact, fünf Dimensionstabellen für kontextuelle Informationen.

2.1. Faktentabelle: sales_fact

Spalte	Typ	Beschreibung
sale_id	UUID	Primärschlüssel
date_id	FK	Zeitdimension
product_id	FK	Produkt
employee_id	FK	Verkäufer
store_id	FK	Filiale
customer_id	FK	Kunde
quantity	int	Anzahl verkaufter Einheiten
unit_price	numeric	Einzelpreis
discount	numeric	Rabatt, default 0
total_amount	numeric	Berechnet: quantity*unit_price - discount

Beschreibung: Jede Zeile repräsentiert ein einzelnes Verkaufsereignis. Die Tabelle verbindet alle Dimensionen für detaillierte Analysen.

2.2. Dimensionstabelle: product_dim

Spalte	Typ	Beschreibung
product_id	UUID	Primärschlüssel
product_name	text	Name des Produkts
category	text	Kategorie
brand	text	Marke
supplier	text	Lieferant

Spalte	Typ	Beschreibung
cost_price	numeric	Einkaufspreis

Beziehung: sales_fact.product_id → product_dim.product_id
Beschreibung: Enthält alle Produktinformationen für analytische Reports.

2.3. Dimensionstabelle: employee_dim

Spalte	Typ	Beschreibung
employee_id	UUID	Primärschlüssel
first_name, last_name	text	Name
role	text	Jobtitel
hire_date	date	Einstellungsdatum
department	text	Abteilung

Beziehungen:

- sales_fact.employee_id → employee_dim.employee_id
- store_dim.manager_id → employee_dim.employee_id

Beschreibung: Mitarbeiter- und Abteilungsinformationen für interne Analysen.

2.4. Dimensionstabelle: store_dim

Spalte	Typ	Beschreibung
store_id	UUID	Primärschlüssel
store_name	text	Name der Filiale
location	text	Stadt/Adresse
region	text	Region

Spalte	Typ	Beschreibung
`manager_id`	`UUID`	Filialleiter

Beziehung: sales_fact.store_id → store_dim.store_id

Beschreibung: Informationen über Stores und Manager.

2.5. Dimensionstabelle: customer_dim

Spalte	Typ	Beschreibung
`customer_id`	UUID	Primärschlüssel
`first_name, last_name`	text	Name des Kunden
`email, phone`	text	Kontakt
`city, country`	text	Standort

Beziehung: sales_fact.customer_id → customer_dim.customer_id

Beschreibung: Kundendaten für Analysen und Segmentierung.

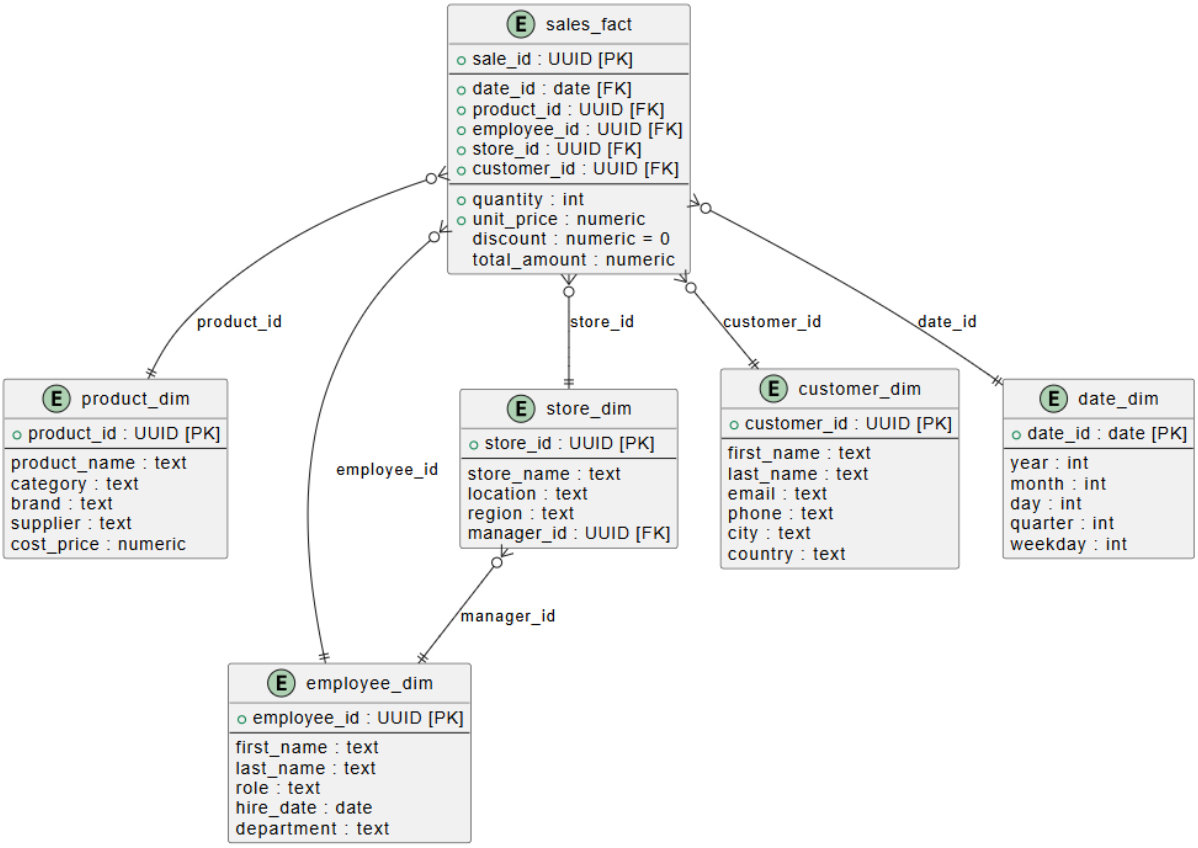
2.6. Dimensionstabelle: date_dim

Spalte	Typ	Beschreibung
`date_id`	`date`	Primärschlüssel
`year, month, day`	`int`	Kalenderinformationen
`quarter`	`int`	Quartal
`weekday`	`int`	Wochentag

Beziehung: sales_fact.date_id → date_dim.date_id
Beschreibung: Unterstützt zeitbasierte Aggregationen (Monats- oder Quartalsumsätze).

2.7. Zusammenfassung ER-Modell

- Zentrale Faktentabelle: sales_fact
- Dimensionen: product_dim, employee_dim, store_dim, customer_dim, date_dim
- Optionaler FK: store_dim.manager_id → employee_dim
- Optimiert für Ad-hoc Queries, Pivot-Tabellen und Drill-Down Analysen



3. LLM-Einsatz

Gemini 2.5 Pro wird verwendet, um natürliche Sprache in SQL-Queries zu übersetzen und die beste Visualisierung für Nivo.js auszuwählen.

3.1. Vorteile von Gemini 2.5 Pro

- Automatische Dimensionserkennung für Star Schema
- Intelligente Chart-Auswahl (Bar, Line, Pie etc.)
- Adaptive Query-Optimierung → reduziert DB-Load
- Predictive Caching → häufige Anfragen können vorhergesagt und gecacht werden

3.2. Einsatzszenarien

- Interaktive Dashboards
- Ad-hoc Analysen
- Automatisierte Report-Erstellung
- Echtzeit-Datenvisualisierung

4. Caching-Strategien in SmartQueryBI

SmartQueryBI implementiert drei Caching-Ebenen, die auf unterschiedlichen Systemebenen arbeiten:

1. Client-Side Cache – Browser/CDN
2. Server-Side LRU Cache – In-Memory auf einer Serverinstanz
3. Redis Distributed Cache – Multi-Instance, persistent

4.1. Client-Side Caching

4.1.1. Funktionsweise

- Nutzt HTTP-Header (Cache-Control) für Browser oder CDN
- Wiederholte identische Requests werden lokal beantwortet
- Reduziert Server- und Datenbanklast

4.1.2. Implementierung

```
response.headers["Cache-Control"] = "public, max-age=60"

result = cached_full_pipeline(request.description)
```

4.1.3. Vorteile

- Sub-millisekunden-Latenz
- Entlastung von Server und DB
- Optimal für Dashboards mit wiederholten Abfragen

4.1.4. Nachteile

- Nur für identische Anfragen effektiv
- Personalisierte Daten müssen als private gecacht werden
- Keine Kontrolle über Browser-Cache-Inhalte

4.1.5. Einsatzszenarien

- Wiederholte KPI-Visualisierungen
- Standardisierte Reports
- Interaktive Frontend-Dashboards

4.2. Server-Side LRU Cache

4.2.1. Funktionsweise

- Nutzt Python functools.lru_cache
- Speichert die letzten n Pipelines im RAM
- Ideal für häufig wiederholte Serveranfragen

4.2.2. Implementierung

```
@lru_cache(maxsize=128)

def cached_full_pipeline(description: str):

    llm_result = generate_visualization_query(description)

    raw_data = execute_llm_select_query(llm_result["sql"]).get("results",
[])
```

```
    nivo_data = generate_nivo_dataset(llm_result["chart"], raw_data)
return {"chart": llm_result["chart"], "llm_query": llm_result["sql"],
"data": nivo_data}
```

4.2.3. Vorteile

- Millisekunden-Antworten
- Kein externer Speicher notwendig
- Geringer Overhead

4.2.4. Nachteile

- Keine Persistenz
- Nur für eine Serverinstanz
- Begrenzter Speicher (maxsize) → alte Einträge werden entfernt

4.2.5. Einsatzszenarien

- Häufige Server-Requests
- Ad-hoc-Analysen
- Vorhalten populärer Reports im RAM

4.3. Redis Distributed Cache

4.3.1. Funktionsweise

- Externer Key-Value Store
- Multi-Server-fähig
- Persistente Zwischenspeicherung

4.3.2. Implementierung

```
cache_key = f"smartquerybi:{hash(request.description)}"
cached = redis_client.get(cache_key)
if cached:
    return json.loads(cached)
```

```
result = cached_full_pipeline(request.description)
redis_client.setex(cache_key, 60, json.dumps(result))
```

4.3.3. Vorteile

- Skalierbar, persistent
- Multi-User geeignet
- Reduziert LLM- und DB-Aufrufe signifikant

4.3.4. Nachteile

- Netzwerkabhängige Latenz
- Redis Deployment & Monitoring notwendig
- Active Cache Invalidation erforderlich

4.3.5. Einsatzszenarien

- Enterprise-Dashboards
- Multi-Server Umgebungen
- Persistente, häufig genutzte Queries

4.4. Vergleich der Caching-Ebenen

Eigenschaft	**Client**	**Server LRU**	**Redis**
Ort	Browser / CDN	Server RAM	Externer Store
Persistenz	Kurzlebig	Nein	Ja
Latenz	sub-ms	ms	1–10 ms
Skalierbarkeit	pro Client	Single Server	Multi-Server
Vorteile	extrem schnell	schnell, gering	persistent, verteilt
Nachteile	Datenlecks	kein Multi-Server	Netzwerk, Setup

Eigenschaft	Client	Server LRU	Redis
Typische Nutzung	Dashboards	Häufige Server-Queries	Shared Queries, Multi-User

4.5. Best Practices

1. Layered Cache-Ansatz:
   Client → Redis → Server LRU → Pipeline
2. TTL bewusst setzen:
   - Client: 30–60 Sekunden
   - Redis: 60 Sekunden (anpassbar)
   - LRU: abhängig von RAM und Nutzung
3. User-Isolation:
   - Personal Data → Cache-Control: private oder user-spezifischer Redis-Key
4. Monitoring:
   - Cache Hit/Miss-Raten
   - Speicherverbrauch
   - Latenzen
5. Adaptive Strategie:
   - Häufige Requests → langer TTL, alle Ebenen
   - Seltene Requests → nur LRU oder Redis, kein Client-Cache

4.6. Architekturdiagramm (Cache-Hierarchie)

User Request

↓

Client-Side Cache (Browser)

↓

Redis Distributed Cache

↓

Server-Side LRU Cache

↓

LLM Pipeline → SQL → DB → Nivo Dataset

5. Fazit

Die Implementierung eines mehrschichtigen Caching-Systems in SmartQueryBI zeigt, wie Performance, Skalierbarkeit und Kosteneffizienz in modernen BI-Plattformen signifikant verbessert werden können. Durch die Kombination von Client-Side Cache, Server-Side LRU Cache und Redis Distributed Cache werden unterschiedliche Anforderungen optimal abgedeckt:

1. Client-Side Cache
   - Reduziert Netzwerklatenz auf nahezu null
   - Entlastet Server und Datenbank bei wiederholten Abfragen
   - Besonders geeignet für Dashboards und sich wiederholende Visualisierungen
2. Server-Side LRU Cache
   - Bietet schnelle, RAM-basierte Antwortzeiten
   - Ideal für häufig wiederholte Anfragen auf einer einzelnen Serverinstanz
   - Geringer Implementierungsaufwand, aber limitiert in Skalierbarkeit und Persistenz
3. Redis Distributed Cache
   - Ermöglicht Multi-Server-Deployments und persistente Zwischenspeicherung
   - Reduziert LLM- und Datenbanklast erheblich
   - Optimal für Enterprise-Szenarien mit hoher Nutzerzahl

Kritische Erfolgsfaktoren:

- Richtige TTL-Konfiguration: Balance zwischen Aktualität und Performance
- User-Isolation: Schutz personenbezogener Daten in allen Caching-Ebenen
- Robuste Invalidation: Sicherstellen, dass bei Datenänderungen der Cache aktualisiert wird
- Monitoring und Analyse: Früherkennung von Performance-Problemen und Optimierung der Cache-Hierarchie

Durch die Integration von Gemini 2.5 Pro als LLM konnte die Plattform semantisch korrekte SQL-Abfragen generieren und die bestmöglichen Visualisierungen automatisch auswählen. Kombiniert mit dem mehrschichtigen Caching-System ergeben sich signifikante Vorteile:

- Performance: Millisekunden- bis Sub-Millisekunden-Antwortzeiten
- Skalierbarkeit: Horizontale Erweiterung durch Redis, Reduzierung der DB-Last
- Kosteneffizienz: Verringerung der LLM-API-Aufrufe und optimierte Serverressourcen
- Benutzerfreundlichkeit: Interaktive Dashboards ohne Wartezeiten

Ein strategisch implementiertes mehrschichtiges Caching-System ist in modernen BI-Plattformen unverzichtbar, um Performance, Kosten und Skalierbarkeit zu optimieren. SmartQueryBI zeigt, dass die Kombination von LLM-basierter SQL-Generierung mit einer intelligenten Cache-Hierarchie eine effiziente, robuste und nutzerfreundliche BI-Lösung ermöglicht.